GenAI Services v2 — Comprehensive Project Documentation

Last updated: 2026-03-19

1) Project Overview

This repository implements a **full-stack AI chatbot builder platform** where users can:

- Sign up / log in (Supabase auth-backed)
- Create **Static Bots** by uploading documents (PDF/TXT/DOCX)
- Create **Dynamic Bots** by scraping website content (including authenticated sites via Selenium login flows)
- Chat with bots through:
  - Frontend app chat widgets
  - Public API endpoint (`/api/chat`)
  - Embeddable website widget script (`frontend/public/widget/widget.js`)
- Manage bots (list, inspect documents, test, generate embed script, delete)

The retrieval + generation pipeline is:

1. Ingest text chunks
2. Embed with `sentence-transformers` (`all-MiniLM-L6-v2`)
3. Store vectors in Qdrant collections (per bot)
4. Retrieve relevant chunks by semantic search
5. Generate final answer with Google Gemini (`gemini-2.5-flash`)

---

2) High-Level Architecture

2.1 Frontend

- Framework: React + TypeScript + Vite + Tailwind CSS
- Routing: `react-router-dom`
- Animation: `framer-motion`
- API client: `axios`

Primary frontend app responsibilities:

- Auth screens and token persistence (`sessionStorage`)
- Bot creation workflows (document upload + website scraping)
- Bot list management and actions
- Chat preview/testing UI
- Widget snippet generation for external websites

2.2 Backend

- Framework: FastAPI

- Auth/DB: Supabase
- Vector DB: Qdrant (local via Docker or cloud URL)
- Embeddings: SentenceTransformers
- LLM: Google Generative AI (Gemini)
- Scraping stack:
  - HTTP: `aiohttp` + `BeautifulSoup`
  - Dynamic/auth fallback: Selenium + headless Chrome

Primary backend responsibilities:

- User auth endpoints
- Bot CRUD and ownership checks
- Document ingestion/chunking/vectorization
- Website crawl/scrape/chunk/vectorization
- Retrieval and LLM response generation
- Health check with Qdrant status

---

3) Repository Structure (Important Paths)

3.1 Root

- `docker-compose.yml` — Qdrant local service
- `setup.ps1` — top-level environment setup helper (Python deps)
- `profile.ps1` — auto-activate venv helper
- `Shopping_Wbsite.html` — standalone demo shopping SPA page (useful as scrape target)
- `PROJECT_DOCUMENTATION.md` — this file

3.2 Backend (`backend/`)

- `app/main.py` — FastAPI app bootstrap, CORS, routers
- `app/core/config.py` — environment settings loader
- `app/api/` — API routers and dependencies
- `app/services/` — auth, chat, vector store, scraping, bot deletion, AI
- `app/utils/document_processor.py` — PDF/TXT/DOCX extraction + chunking + widget code generator
- `requirements.txt` — Python dependencies
- `render.yaml`, `Procfile`, `runtime.txt` — deployment metadata
- `setup_backend.*` + `run_backend.*` — setup/run scripts for Windows (BAT + PowerShell)
- `migrate_to_qdrant.py` — FAISS→Qdrant migration utility
- `test_api.py`, `test_qdrant.py`, `test_doc.txt` — validation scripts/sample doc
- docs: `START_HERE.md`, `QUICK_START.md`, `PERMANENT_FIX.md`, `UNDERSTANDING_VENV.md`, `CHATBOT_FIXES.md`

3.3 Frontend (`frontend/`)

- `src/App.tsx` — route setup + protected layout shell
- `src/context/AuthContext.tsx` — auth state + login/logout behavior
- `src/lib/api.ts` — Axios instance + auth interceptors

- `src/lib/botApi.ts` — bot upload/scrape/chat/delete calls
- `src/pages/` — Login, SignUp, Dashboard, CreateBot, BotsList
- `src/components/` — Navigation, ChatWidget, Modal
- `public/widget/widget.js` — embeddable external chatbot widget
- `vite.config.ts`, `tailwind.config.js`, `vercel.json` — build/deploy/frontend tooling

---

4) Backend Deep-Dive

4.1 App Initialization

File: `backend/app/main.py`

- FastAPI app title: `Chatbot Builder API`
- Custom JSON response encoder to serialize `datetime`
- CORS: `allow_origins=["*"]`, all methods/headers enabled, credentials disabled
- Routers:
  - `/auth` → auth endpoints
  - `/api` → main business endpoints

4.2 Configuration

File: `backend/app/core/config.py`

Environment-driven settings include:

- Project metadata (`PROJECT_NAME`, `VERSION`)
- JWT secrets/algorithm/expiry settings
- Supabase URL + anon key
- Google API key
- Qdrant host/port/API key/cloud URL
- Frontend URL

`.env` is loaded via `python-dotenv`.

4.3 API Endpoints

File: `backend/app/api/endpoints.py`

Bot and document endpoints (authenticated)

- `GET /api/bots`
  - Returns current user's bots (ownership-scoped)
- `GET /api/bots/stats`
  - Returns placeholder aggregate stats currently hardcoded
- `DELETE /api/bots/{bot_id}`
  - Deletes bot and attempts Qdrant collection cleanup
- `GET /api/bots/{bot_id}/documents`
  - Verifies ownership, then returns grouped document/chunk metadata

Ingestion endpoints

- `POST /api/upload` (multipart form)
  - Inputs:
    - `company_name`
    - `files[]` (`.pdf`, `.txt`, `.docx` allowed)
  - Flow:
    1. Validate form + file types
    2. Create bot in Supabase
    3. Save temp files
    4. Extract + chunk text
    5. Vectorize + store in Qdrant
    6. Return bot id + generated widget code
- `POST /api/scrape`
  - Body: `company_name`, `website_url`, optional login credentials + role
  - Flow:
    1. Crawl/scrape website
    2. Create bot
    3. Build metadata (`web:<source_url>` filenames)
    4. Store chunks/vectors
    5. Return bot id + scrape stats + widget code

Chat endpoints

- `POST /api/chat` (public, no auth required)
  - Body: `{ bot_id, query }`
  - Supports anonymous widget/public chat
- `POST /api/bots/{bot_id}/chat` (authenticated)
  - Body: `{ query }`
  - Verifies user access to bot

Health endpoint

- `GET /api/health`
  - API health + Qdrant health and status aggregation

4.4 Auth Endpoints

File: `backend/app/api/auth.py`

- `POST /auth/register`
  - Creates user via Supabase
  - Returns user data
  - Sets secure HTTP-only session cookie when available
- `POST /auth/login`

    - Authenticates user
    - Returns user + token pair + bots list
    - Sets session cookie if present

- `POST /auth/logout`
    - Signs out (best-effort) + clears cookie

4.5 Auth Dependency Resolution

File: `backend/app/api/dependencies.py`

`get_current_user` supports multiple auth input channels:

1. Bearer token via `Authorization` header
2. OAuth2 dependency token
3. Session cookie fallback

Then `get_current_active_user` ensures user presence.

4.6 Core Services

`AuthService` (`backend/app/services/auth.py`)

- Singleton service with shared Supabase client
- Sign up / sign in wrappers
- Stateless token/session verification (`get_user(token)` style)
- User bot fetching from `bot_info`
- Bot creation via Supabase RPC (`create_bot`)
- Includes retries and RLS-related error handling

`BotService` (`backend/app/services/bot.py`)

- Verifies user bot ownership
- Deletes Qdrant collection (`bot_{bot_id}`)
- Deletes Supabase `bot_info` row

`VectorStoreService` (`backend/app/services/vector_store.py`)

- Singleton Qdrant + embedding model manager
- Model: `all-MiniLM-L6-v2` (384 dim)
- Supports:
    - Collection create/delete/list/info/stats
    - Batched text embedding
    - Batched upserts with retries + exponential backoff
    - Search via `query_points` with legacy fallback to `.search`
    - Scroll, payload update, point deletion
    - Health check

`ChatService` (`backend/app/services/chat.py`)

- Ownership verification (or anonymous bypass for public endpoint)
- Document retrieval for bot display
- Chat response generation flow:
    1. Search top results in bot collection
    2. Multi-phase relevance thresholds (`0.15` then `0.10` then broader search)
    3. Build context (deduplicated chunks)
    4. Prompt Gemini with strict response behavior rules
    5. Return generated answer / robust fallback message
- Document processing pipeline to vector store in batches

`AIService` (`backend/app/services/ai_service.py`)

- Lazy initialization of Gemini model
- Uses `GOOGLE_API_KEY`
- Generates completion using `gemini-2.5-flash`

`WebScraperService` (`backend/app/services/web_scraper.py`)

Implements two-tier scraping:

1. **HTTP crawl first** (`aiohttp` + BeautifulSoup)
2. **Selenium fallback** for JS-heavy/SPA or authenticated pages

Key capabilities:

- Same-domain crawl
- Internal link discovery
- JS-rendered content capture
- Optional login support with multiple strategies:
    - Standard single-page login
    - Multi-step login
    - Role-card selection (e.g., Student/Employee)
    - Popup/cookie dismissal
- Structured extraction from body/template/script data
- Chunking with metadata (`source_url`, `page_title`, `source_type`)

4.7 Document Processor

File: `backend/app/utils/document_processor.py`

- Parses PDF (`PyPDF2`), DOCX (`python-docx`), TXT (`chardet` encoding detection)
- Normalizes text and splits with `RecursiveCharacterTextSplitter`
- Default chunking:
    - `chunk_size=1000`
    - `chunk_overlap=200`
- Also generates embed script snippet (`generate_widget_code`)

---

5) Frontend Deep-Dive

5.1 Routing and Layout

File: `frontend/src/App.tsx`

- Uses `BrowserRouter`
- Public routes: `/login`, `/signup`
- Protected routes:
  - `/dashboard`
  - `/create-bot`
  - `/bots`
- Shared authenticated shell:
  - left sidebar (`Navigation`)
  - main content container (`max-w-[1200px]`)

5.2 Authentication State

File: `frontend/src/context/AuthContext.tsx`

- Initializes user from `sessionStorage` synchronously to avoid route flicker
- `login` clears stale session first, then stores `token` + `user`
- `logout` clears session and redirects to `/login`

5.3 API Layer

`frontend/src/lib/api.ts`

- Base URL from `VITE_API_URL` (default `http://localhost:8000`)
- Request interceptor injects `Authorization: Bearer <token>`
- Response interceptor handles 401/403 by clearing session + redirecting

`frontend/src/lib/botApi.ts`

- `createBot(formData)` → `POST /api/upload`
- `createDynamicBot(...)` → `POST /api/scrape`
- `sendChatMessage(botId, message)` → `POST /api/chat`
- `deleteBot(botId)` → `DELETE /api/bots/{bot_id}`

5.4 Pages and Features

Login (`frontend/src/pages/Login.tsx`)

- Email/password login form
- Calls `useAuth().login`
- Redirects to dashboard on success

SignUp (`frontend/src/pages/SignUp.tsx`)

- Email/password/confirm flow
- Client-side validation (email format, min length, match)
- Calls `/auth/register`

- Shows verification message and redirects to login

Dashboard (`frontend/src/pages/Dashboard.tsx`)

- Fetches bot count from `/api/bots`
- Displays platform overview cards:
  - total bots
  - AI engine
  - vector store
- Highlights capability list and quick action links

Create Bot (`frontend/src/pages/CreateBot.tsx`)

Two modes:

1. **Static Bot**
   - Bot name + PDF upload UX (drag/drop + picker)
   - Calls upload API
2. **Dynamic Bot**
   - Website URL scraping flow
   - Optional authenticated crawl fields:
     - login URL
     - username
     - password
     - role
   - Security notice + explicit user acknowledgment
   - Calls scrape API

After creation:

- Shows success state
- Shows scrape stats for dynamic mode
- Enables embedded `ChatWidget` preview/testing

Bots List (`frontend/src/pages/BotsList.tsx`)

- Fetches bots and per-bot document summaries
- Search/filter by bot name
- Row actions:
  - test chat (modal)
  - view widget embed snippet
  - delete bot (confirm modal)

5.5 Shared Components

`Navigation.tsx`

- Sidebar nav with active state

- Dashboard / Create Bot / My Bots
- Sign out action

`ChatWidget.tsx`

- Message thread UI
- Sends chat messages to backend
- Typing indicator and error fallback message

`Modal.tsx`

- Basic reusable overlay modal container

5.6 External Embeddable Widget

File: `frontend/public/widget/widget.js`

Features implemented:

- Floating launcher button
- Expand/collapse chat window
- Draggable chat panel
- Theme toggle (light/dark)
- Size cycling (small/medium/large)
- Markdown-like formatting in bot messages (`**bold**`, bullets)
- Sends requests to `http://localhost:8000/api/chat`

Script attributes used:

- `data-bot-id`
- `data-company-name`
- `data-color`

---

6) Data Model and Storage

6.1 Supabase

Primary logical table used in code:

- `bot_info` (queried for user bots; deleted by `bot_id`)

Bot creation currently uses RPC:

- `create_bot(p_bot_id, p_user_id, p_name, p_created_at)`

6.2 Qdrant

- Collection naming convention: `bot_<bot_uuid>`
- Point payload includes fields like:
  - `text`

    - `created_at`
    - `bot_id`, `user_id`
    - `filename`
    - `chunk_index`, `chunk_length`
    - optional source metadata for scraped pages

6.3 Migration Assets

- `backend/migrations/create_bots_table.sql` — currently empty
- `backend/migrations/create_document_info_table.sql` — currently empty
- `backend/migrate_to_qdrant.py` — migrates legacy FAISS `.meta`/`.index` data to Qdrant

---

7) Security and Auth Notes

- Frontend stores bearer token in `sessionStorage`
- Backend supports bearer token and session cookie auth paths
- Public chat endpoint intentionally allows anonymous access with known `bot_id`
- CORS is currently permissive (`*`) for widget compatibility
- Dynamic scrape credentials are accepted over API body and intended for in-memory use during scrape session

Important hardening opportunities:

- Restrict CORS in production
- Move widget backend URL to configurable env in widget script
- Rotate/remove real-looking keys in `.env.example` files
- Add stricter rate limiting and abuse protection on public `/api/chat`

---

8) Setup and Run

8.1 Backend (Windows scripts)

From `backend/`:

- First-time setup:
    - `setup_backend.bat` or `./setup_backend.ps1`
- Run server:
    - `run_backend.bat` or `./run_backend.ps1`
- Direct command:
    - `uvicorn app.main:app --reload`

8.2 Frontend

From `frontend/`:

- `npm install`
- `npm run dev`
- `npm run build`

- `npm run preview`

Vite dev proxy forwards `/api` to `http://localhost:8000`.

8.3 Qdrant Local via Docker

From repo root:

- `docker-compose up -d qdrant`

Exposed ports:

- `6333` HTTP
- `6334` gRPC

---

9) Environment Variables

9.1 Backend (`backend/.env`)

Expected keys (from code + `.env.example`):

- `GOOGLE_API_KEY`
- `JWT_SECRET`
- `FRONTEND_URL`
- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY`
- `QDRANT_HOST`
- `QDRANT_PORT`
- `QDRANT_API_KEY`
- `QDRANT_URL` (for cloud mode)

9.2 Frontend (`frontend/.env`)

- `VITE_API_URL`
- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY`

---

10) Deployment Configuration

10.1 Backend

- Render config: `backend/render.yaml`
  - Python 3.11
  - `pip install -r requirements.txt`
  - `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Procfile for process-based platforms present

10.2 Frontend

- Vercel config: `frontend/vercel.json`
  - framework `vite`
  - output `dist`
  - SPA rewrites to `index.html`

---

11) Testing and Diagnostics Utilities

- `backend/test_qdrant.py`
  - Full Qdrant integration verification (create/search/stats/delete)
- `backend/test_api.py`
  - Endpoint sanity checks (`/api/health`, upload accessibility, chat endpoint)
- `backend/test_doc.txt`
  - Sample ingest content for local tests
- `backend/download_model.py`
  - Pre-download embedding model

---

12) Active vs Legacy / Placeholder Files

The codebase includes several files that are present but not active in current route wiring:

- Empty placeholder files:
  - `backend/app/api/me.py`
  - `backend/app/api/auth_new.py`
  - `backend/app/api/dependencies_new.py`
  - `backend/app/services/auth_new.py`
  - `backend/app/services/chat_updated.py`
  - `backend/app/services/vector_store_updated.py`
  - `backend/app/services/get_response.py`
  - `backend/app/services/recovery.py`
  - `frontend/src/lib/botsApi.ts`
  - `frontend/src/components/AuthForm.tsx`
  - `frontend/src/components/Login.tsx`
  - `frontend/src/components/ProtectedRoute.tsx`
- Historical backup/alternate implementations:
  - `backend/app/services/chat.py.bak`
  - `frontend/src/pages/CreateBot.tsx.new`

These are useful for reference but should be treated as non-authoritative unless re-integrated.

---

13) Known Functional Characteristics and Limits

- Public chat endpoint accepts any valid `bot_id` (by design for embeddable widgets)
- Bot stats endpoint currently returns placeholder values
- Widget script API URL is hardcoded to localhost
- SQL migration files for schema creation are currently empty placeholders
- Some deployment/setup docs are duplicated across multiple markdown files in `backend/`

14) End-to-End Feature Matrix

User/Auth

- Register user account ✅
- Login with token response ✅
- Session cleanup/logout ✅

Bot Lifecycle

- Create static bot from files ✅
- Create dynamic bot by crawl/scrape ✅
- List bots per user ✅
- Delete bot + vector cleanup ✅
- View bot documents (grouped by filename) ✅

AI/RAG

- Chunk text with overlap ✅
- Generate embeddings ✅
- Store/search vectors in Qdrant ✅
- Multi-phase threshold retrieval ✅
- Gemini answer generation ✅

Chat Interfaces

- In-app chat widget component ✅
- Public API chat for embeds ✅
- Embeddable website script widget ✅

Ops/Tooling

- Local Qdrant Docker compose ✅
- Render backend config ✅
- Vercel frontend config ✅
- Setup/run scripts for Windows ✅
- Migration/test scripts ✅

---

15) Quick Start (Practical)

1. Start Qdrant:
   - `docker-compose up -d qdrant`
2. Backend:
   - `cd backend`
   - `setup_backend.bat` (first time)
   - `run_backend.bat`
3. Frontend:

    - `cd frontend`
    - `npm install`
    - `npm run dev`
4. Open app:
    - `http://localhost:5173`

---

16) Recommendations for Next Iteration

1. Add real DB migrations in `backend/migrations/*.sql`
2. Externalize widget backend URL (environment-configurable)
3. Add rate limiting + request throttling to public chat endpoint
4. Remove/clean obsolete placeholder files to reduce maintenance confusion
5. Add automated tests for API and critical frontend flows
6. Add production-safe `.env.example` values (no real tokens)

---

17) Version Notes

This documentation reflects the current implementation in the workspace at the time of writing, including:

- Qdrant `query_points` based search path
- adaptive multi-phase retrieval thresholds in chat service
- tightened frontend page containers and spacing alignment updates

If code changes, regenerate/update this document to keep behavior references accurate.